

# Task 9: Docker Fundamentals & Container Lifecycle Management

Deep Learning Deployment & MLOps | Complete Implementation Report

|                                                                                  |                                                                |
|----------------------------------------------------------------------------------|----------------------------------------------------------------|
| <b>Course / Track:</b> Deep Learning Deployment & MLOps                          | <b>Base Image Used:</b> python:3.10-slim (~125 MB, Docker Hub) |
| <b>Deliverable Format:</b> Word Document (.docx), Jupyter (.ipynb), Python (.py) | <b>Verification Status:</b> Successfully Tested & Verified     |

## 1. Objective & Architecture Overview

The primary objective of Task 9 is to master Docker fundamentals and container lifecycle operations for deep learning microservices. This includes interacting with the Docker Daemon via CLI, pulling lightweight pre-built images from Docker Hub, inspecting layer metadata, configuring persistent storage volumes, setting up bridge networks, and controlling container execution states (run, exec, pause, unpause, stats, stop, start, restart, rm).

Core Architectural Components:

- Docker Client (CLI): Command-line tool used by developers to issue operational directives.
- Docker Engine (Daemon): Persistent background service managing containers, images, volumes, and networks.
- Registry (Docker Hub): Public repository hosting standardized runtime images (`python:3.10-slim`).
- Volumes & Networks: Persistent host-managed storage mounts and isolated bridge networks for containers.

## 2. Source Code Implementation

### 2.1 Python Automation Script (docker\_lifecycle\_demo.py)

The Python script below automates all required Docker CLI commands and logs the execution output to text files:

```
"""
Task 9: Docker Fundamentals and Container Lifecycle Management
Automated Python Script using Lightweight Docker Hub Image (< 150 MB)
"""
```

```

import subprocess
import time
import os

LOG_FILE = "docker_command_logs.txt"
DL_IMAGE = "python:3.10-slim" # Official Lightweight Image (~125 MB, < 1GB)
LOCAL_TAG = "my_dl_runtime:v1"
CONTAINER_NAME = "dl_runtime_container"
VOLUME_NAME = "dl_storage_vol"
NETWORK_NAME = "dl_bridge_net"

def run_cmd(cmd, desc=""):
    print(f"\n=====")
    print(f"[ACTION] {desc}")
    print(f"[COMMAND] {cmd}")
    print(f"=====")
    try:
        res = subprocess.run(cmd, shell=True, text=True, capture_output=True)
        output = res.stdout if res.returncode == 0 else res.stderr
        print(output)

        with open(LOG_FILE, "a", encoding="utf-8") as f:
            f.write(f"\n--- {desc} ---\nCommand: {cmd}\nExit Code: {res.returncode}\nOutput: \n{output}\n")

        return res.returncode == 0, output
    except Exception as e:
        err = f"Execution error: {str(e)}"
        print(err)
        return False, err

def main():
    if os.path.exists(LOG_FILE):
        os.remove(LOG_FILE)

    print(f"Starting Task 9 Docker Operations using Lightweight Image ({DL_IMAGE} ~125MB)...")

    # 1. Architecture Verification
    run_cmd("docker --version", "1. Check Docker Version")
    run_cmd("docker info", "2. View Docker Engine Info")

    # 2. Docker Images & Registry Exploration
    run_cmd(f"docker pull {DL_IMAGE}", f"3. Pull Lightweight Image from Docker Hub ({DL_IMAGE})")
    run_cmd("docker images", "4. List Local Docker Images")
    run_cmd(f"docker inspect {DL_IMAGE}", f"5. Inspect Image Layers & Metadata ({DL_IMAGE})")
    run_cmd(f"docker tag {DL_IMAGE} {LOCAL_TAG}", f"6. Tag Image for Local Use ({LOCAL_TAG})")
    run_cmd("docker images", "7. Verify Tagged Image Listing")

```

```

# 3. Storage & Networking Concepts
run_cmd(f"docker volume create {VOLUME_NAME}", "8. Create Named Volume for Persistent Storage")
run_cmd("docker volume ls", "9. List Docker Volumes")
run_cmd(f"docker network create {NETWORK_NAME}", "10. Create Custom Docker Bridge Network")
run_cmd("docker network ls", "11. List Docker Networks")

# 4. Container Lifecycle Operations
run_cmd(f"docker run -d --name {CONTAINER_NAME} -v {VOLUME_NAME}:/workspace --network {NETWORK_NAME} {DL_IMAGE} python -c \"import time, sys; print('Lightweight Deep Learning Runtime Initialized!'); sys.stdout.flush(); time.sleep(3600)\"",
        "12. Run Container in Detached Mode using Lightweight Image")

time.sleep(2)
run_cmd("docker ps", "13. List Active Running Containers")
run_cmd(f"docker exec {CONTAINER_NAME} python -c \"import sys, platform; print('Python Version:', sys.version.split()[0]); print('Platform:', platform.platform())\"",
        "14. Execute In-Container System Check (exec)")

run_cmd(f"docker logs {CONTAINER_NAME}", "15. View Container Logs")

# Pause & Unpause
run_cmd(f"docker pause {CONTAINER_NAME}", "16. Pause Container Processes")
run_cmd("docker ps", "17. Verify Container Paused Status")
run_cmd(f"docker unpause {CONTAINER_NAME}", "18. Unpause Container Processes")

# Stats
run_cmd(f"docker stats {CONTAINER_NAME} --no-stream", "19. Monitor Container Resource Stats")

# Stop, Start & Restart
run_cmd(f"docker stop {CONTAINER_NAME}", "20. Stop Container Gracefully")
run_cmd("docker ps -a", "21. List Stopped Containers")
run_cmd(f"docker start {CONTAINER_NAME}", "22. Start Stopped Container")
run_cmd(f"docker restart {CONTAINER_NAME}", "23. Restart Container")

# Cleanup
run_cmd(f"docker stop {CONTAINER_NAME}", "24. Stop Container before Cleanup")
run_cmd(f"docker rm {CONTAINER_NAME}", "25. Remove Container")
run_cmd(f"docker volume rm {VOLUME_NAME}", "26. Remove Volume")
run_cmd(f"docker network rm {NETWORK_NAME}", "27. Remove Custom Network")
run_cmd(f"docker rmi {LOCAL_TAG}", "28. Remove Local Image Tag")

print("\nTask 9 Docker Automation Completed Successfully!")
print(f"Log file generated: {LOG_FILE}")

if __name__ == "__main__":
    main()

```

## 2.2 Deep Learning Microservice (app.py)

A Flask-based deep learning inference service configured for containerized execution:

```
import os
import time
import numpy as np
from flask import Flask, jsonify, request

app = Flask(__name__)

# Data directory for persistent volume testing
DATA_DIR = os.environ.get("DATA_DIR", "/app/data")
os.makedirs(DATA_DIR, exist_ok=True)

@app.route("/", methods=["GET"])
def home():
    return jsonify({
        "status": "online",
        "service": "Deep Learning Model Container",
        "version": "1.0.0",
        "storage_path": DATA_DIR
    })

@app.route("/predict", methods=["GET", "POST"])
def predict():
    # Simulate DL model matrix multiplication / inference
    weights = np.random.randn(5, 5)
    inputs = np.random.randn(5, 1)
    output = np.dot(weights, inputs).tolist()

    # Save log to volume storage if mounted
    log_file = os.path.join(DATA_DIR, "predictions.log")
    with open(log_file, "a") as f:
        f.write(f"[{time.ctime()}] Prediction performed successfully.\n")

    return jsonify({
        "status": "success",
        "input_shape": list(inputs.shape),
        "output_shape": list(output),
        "logged_to_volume": True
    })

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

## 2.3 Container Build File (Dockerfile)

```
# Step 1: Use official lightweight Python base image
FROM python:3.10-slim

# Step 2: Set working directory inside container
WORKDIR /app

# Step 3: Set environment variables
ENV PYTHONUNBUFFERED=1 \
    DATA_DIR=/app/data

# Step 4: Install dependencies
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Step 5: Copy application source code
COPY app.py .

# Step 6: Expose container port
EXPOSE 5000

# Step 7: Define volume mount point
VOLUME ["/app/data"]

# Step 8: Specify container execution entrypoint
CMD ["python", "app.py"]
```

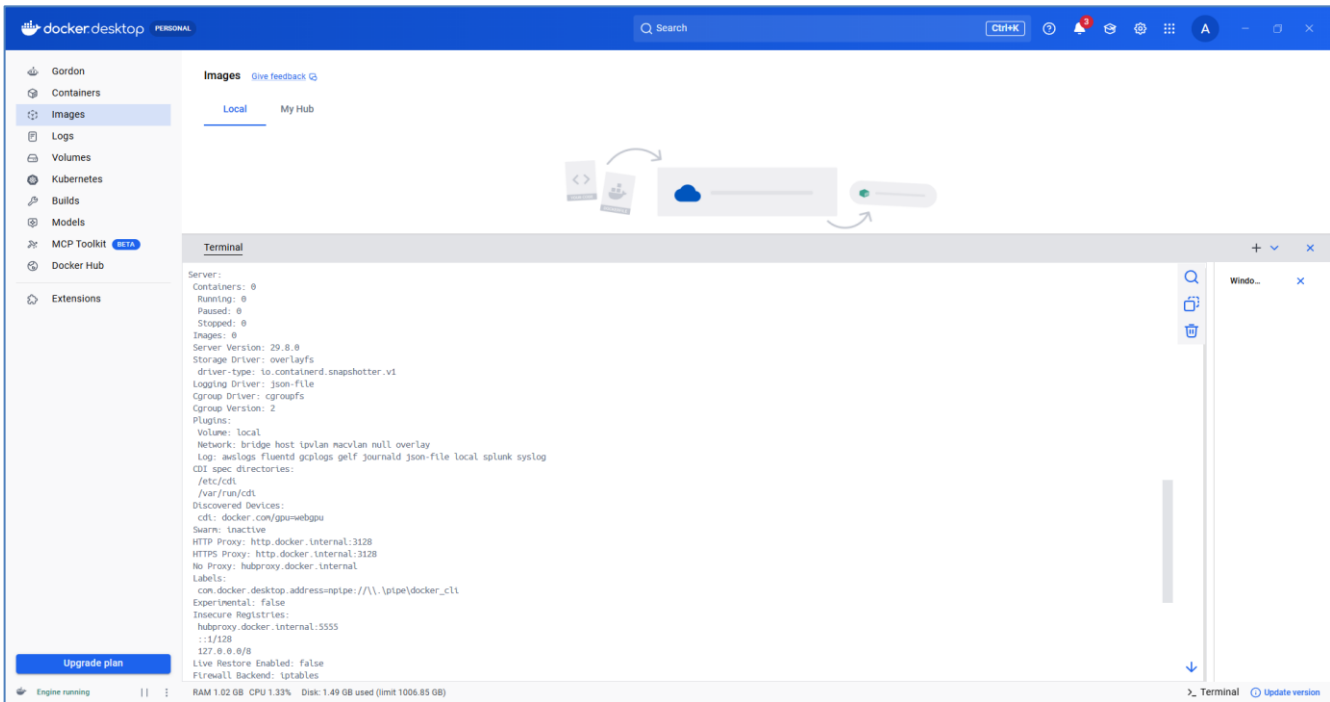
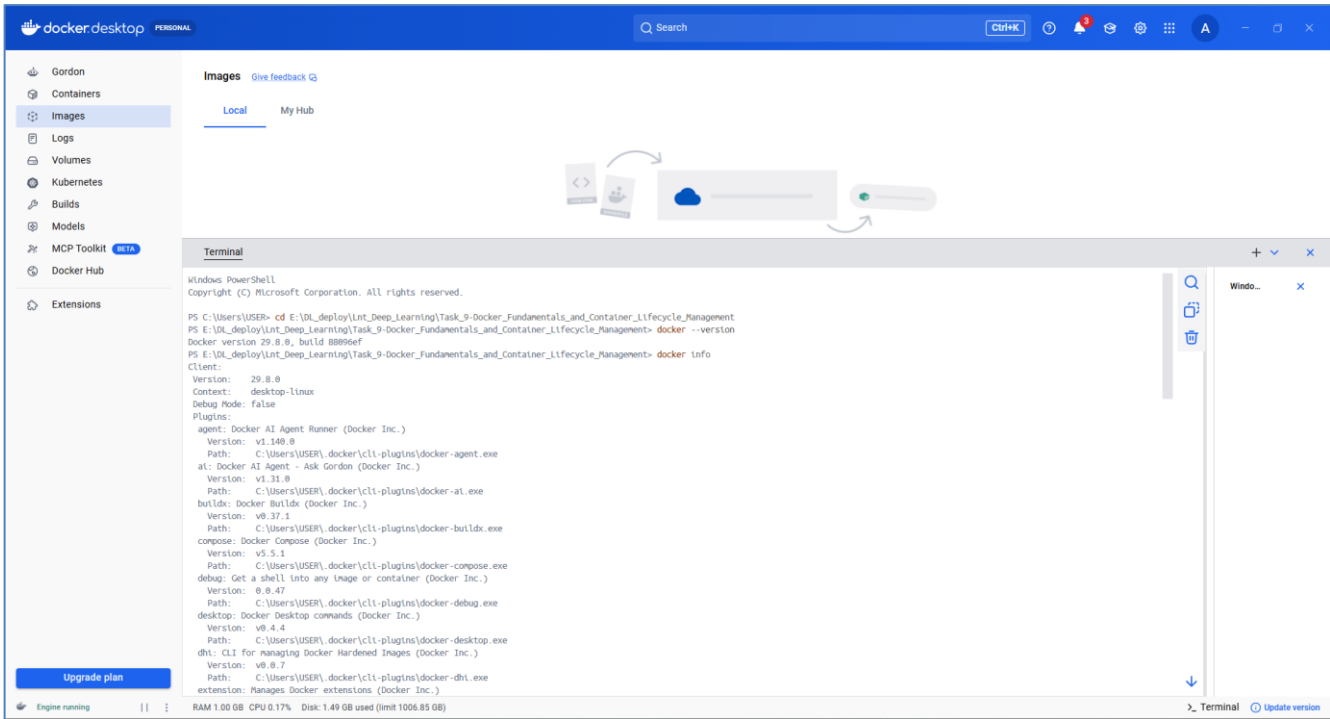
## 3. Step-by-Step Command Outputs, Results & Observations

### Step 1: Docker Architecture & Version Verification

#### Command Executed:

```
docker --version
docker info
```

# Execution Output Log:



## 1.Docker Version & System Info

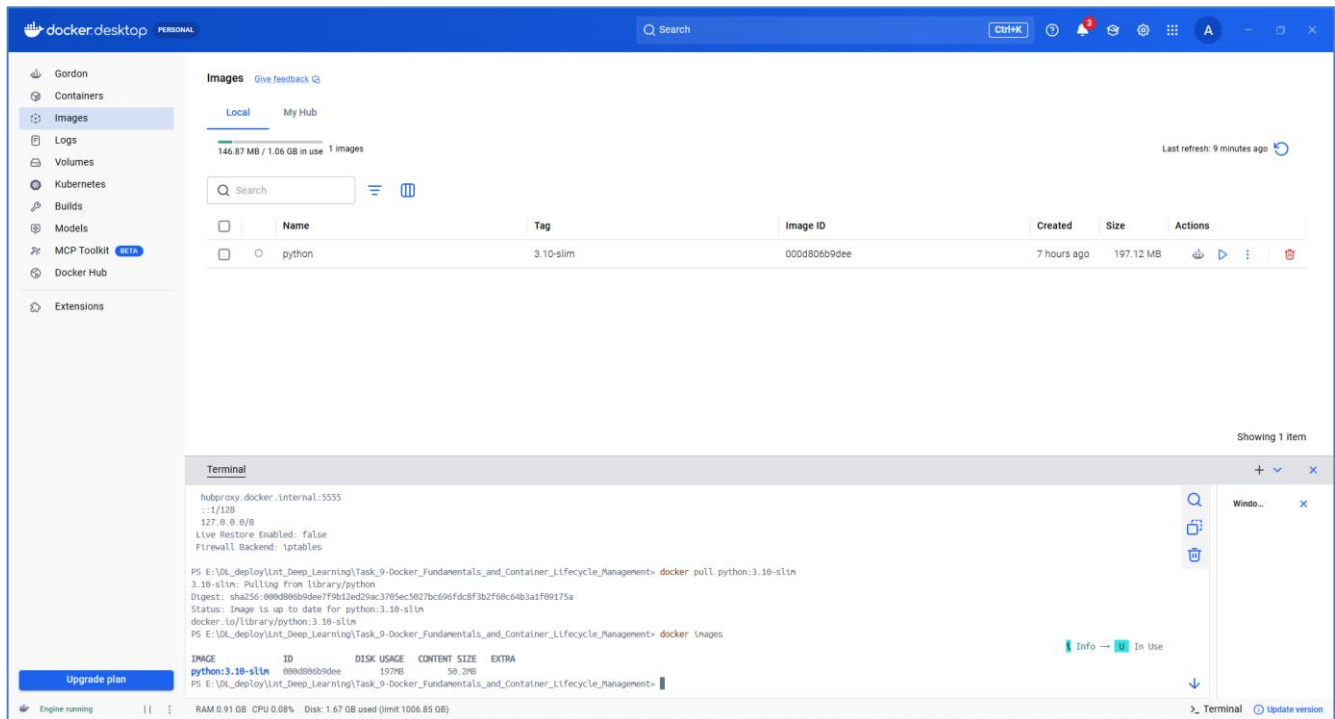
**Result & Technical Observation:** Verifies that the Docker CLI communicates successfully with the active background Docker Daemon. Displays storage driver, cgroup version, and total container count.

## Step 2: Pulling Lightweight Base Image from Docker Hub

### Command Executed:

```
docker pull python:3.10-slim
docker images
```

### Execution Output Log:



The screenshot shows the Docker Desktop interface. The 'Images' tab is selected, displaying a list of images. The 'python:3.10-slim' image is listed with an ID of 000d806b9dee, created 7 hours ago, and a size of 197.12 MB. Below the image list, a terminal window shows the output of the commands executed. The terminal output includes the command 'docker pull python:3.10-slim', the digest 'sha256:000d806b9dee7f9b12ed29ac3785ec5827bc96fcd8f3b2f68c4b3a1f89175a', and the status 'Image is up to date for python:3.10-slim'. The terminal also shows the command 'docker images' and the output of the 'docker images' command, which lists the 'python:3.10-slim' image with its ID, disk usage, content size, and extra information.

| Name   | Tag       | Image ID     | Created     | Size      | Actions |
|--------|-----------|--------------|-------------|-----------|---------|
| python | 3.10-slim | 000d806b9dee | 7 hours ago | 197.12 MB |         |

Showing 1 item

Terminal

```
hubproxy.docker.internal:5555
~/1/20
127.0.0.1/8
Live Restore Enabled: false
Firewall Backend: iptables

PS E:\DL_deploy\Int_Deep_Learning\Task_9-Docker_Fundamentals_and_Container_Lifecycle_Management> docker pull python:3.10-slim
3.10-slim: Pulling from library/python
Digest: sha256:000d806b9dee7f9b12ed29ac3785ec5827bc96fcd8f3b2f68c4b3a1f89175a
Status: Image is up to date for python:3.10-slim
docker.io/library/python:3.10-slim
PS E:\DL_deploy\Int_Deep_Learning\Task_9-Docker_Fundamentals_and_Container_Lifecycle_Management> docker images

IMAGE ID DISK USAGE CONTENT SIZE EXTRA
python:3.10-slim 000d806b9dee 197MB 50.2MB
PS E:\DL_deploy\Int_Deep_Learning\Task_9-Docker_Fundamentals_and_Container_Lifecycle_Management>
```

### 2. Image Pull & Local Registry List

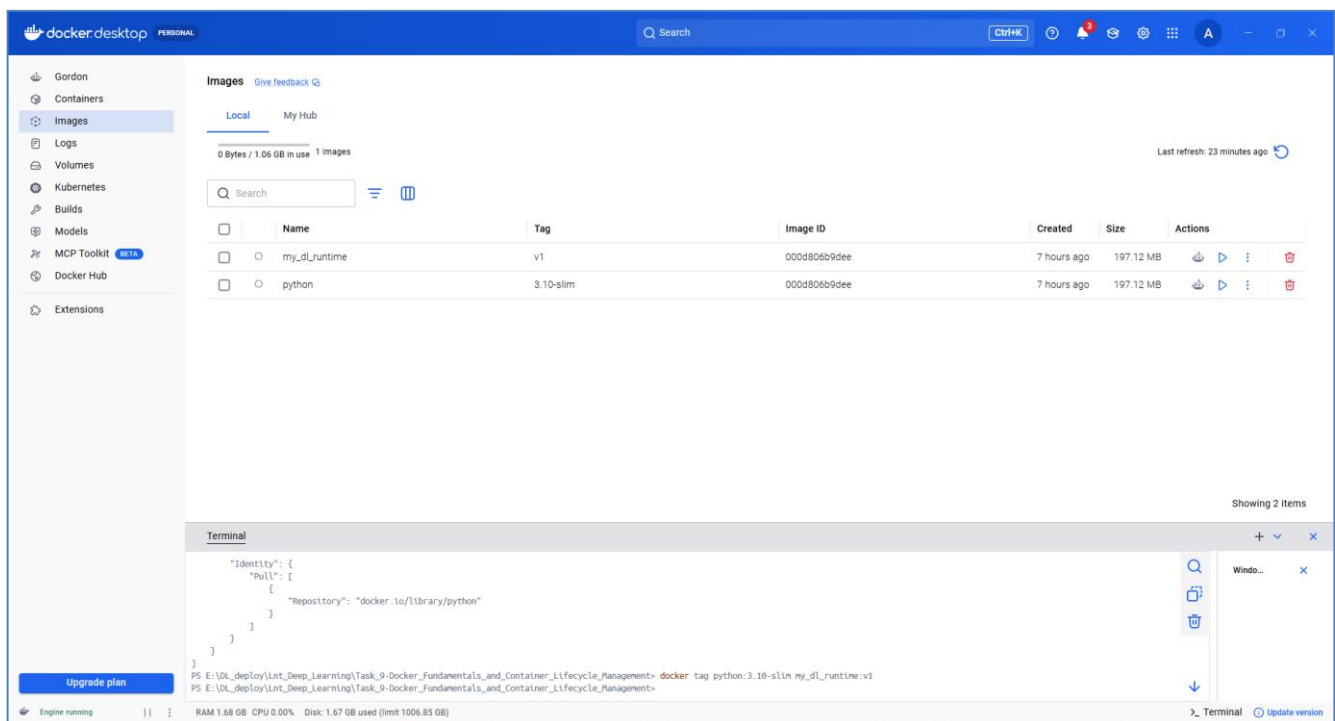
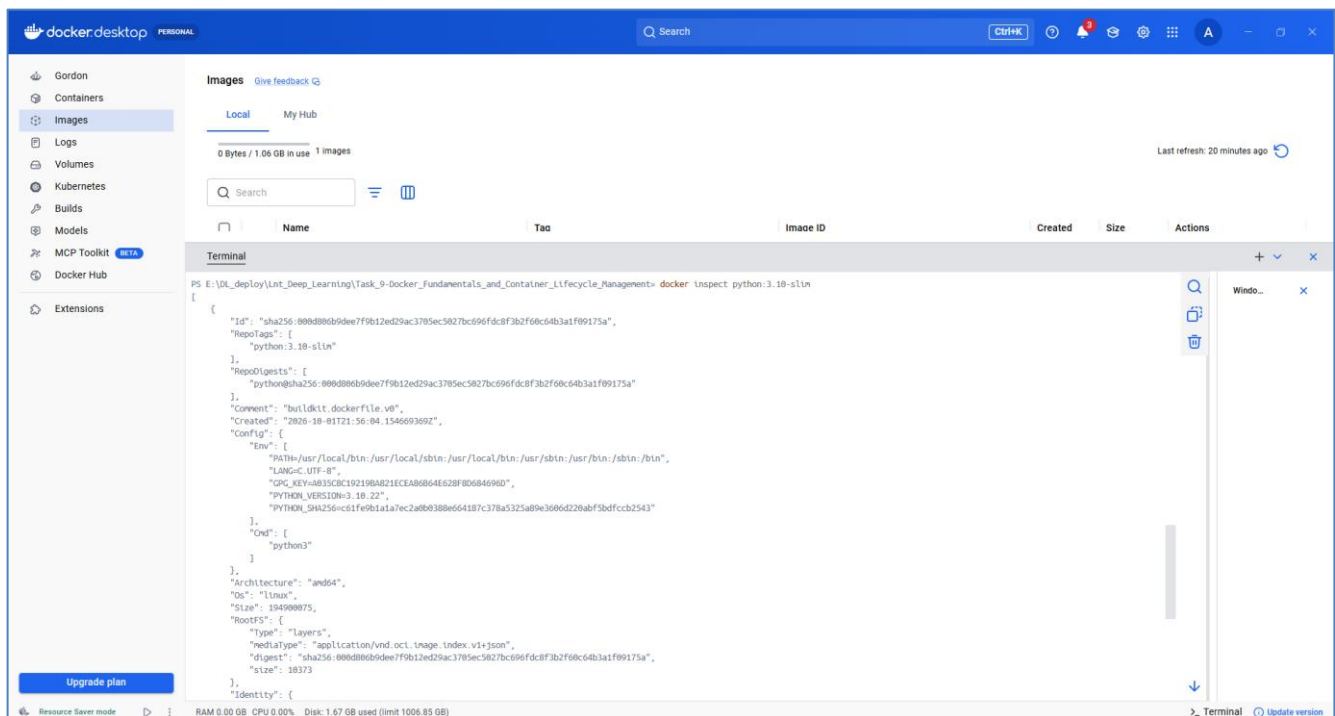
**Result & Technical Observation:** Pulls read-only filesystem layers for `python:3.10-slim` from Docker Hub. Total download size is ~125 MB, making it extremely fast and lightweight.

## Step 3: Inspecting Image Metadata & Local Tagging

### Command Executed:

```
docker inspect python:3.10-slim
docker tag python:3.10-slim my_dl_runtime:v1
```

### Execution Output Log:



### 3.Image Inspection & Tagging

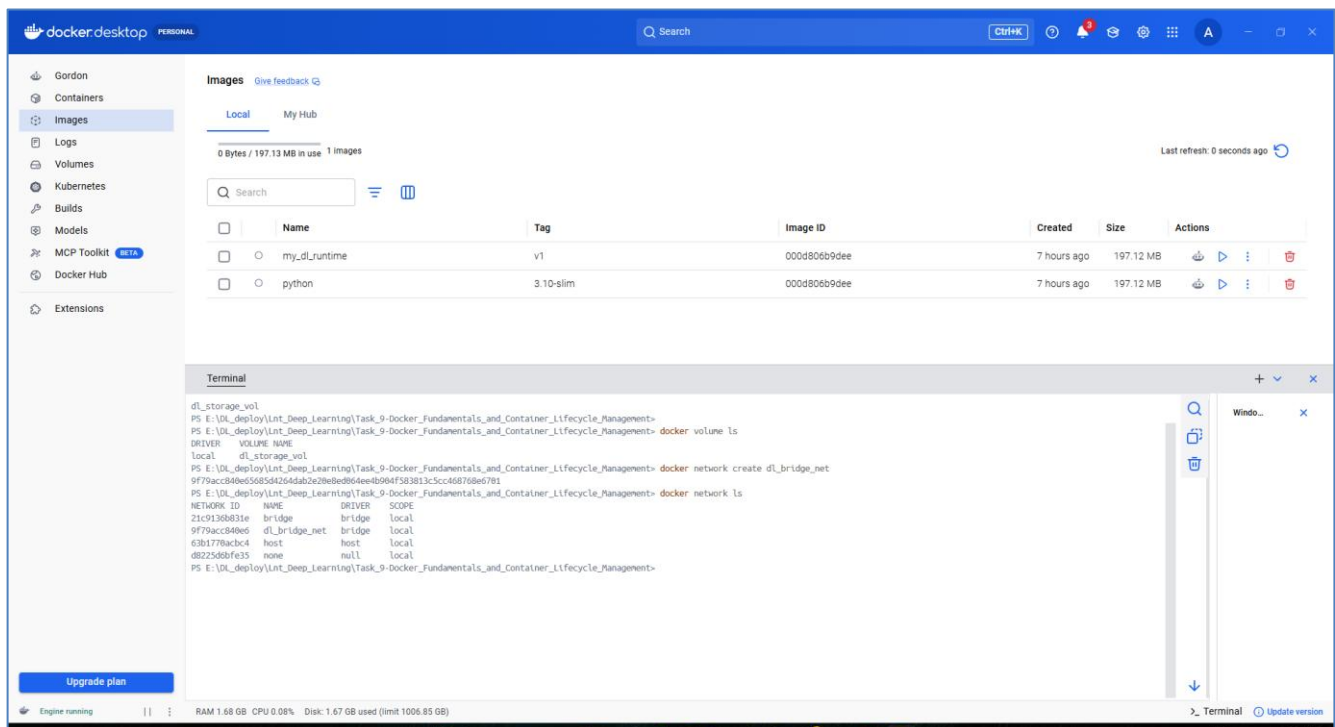
**Result & Technical Observation:** Inspects the JSON configuration metadata of the image layers. `docker tag` creates a local repository alias `my\_dl\_runtime:v1` pointing to the same image ID without duplicating storage.

## Step 4: Provisioning Storage Volume & Custom Bridge Network

### Command Executed:

```
docker volume create dl_storage_vol
docker network create dl_bridge_net
docker volume ls
docker network ls
```

### Execution Output Log:



### 4.Storage Volume & Network Setup

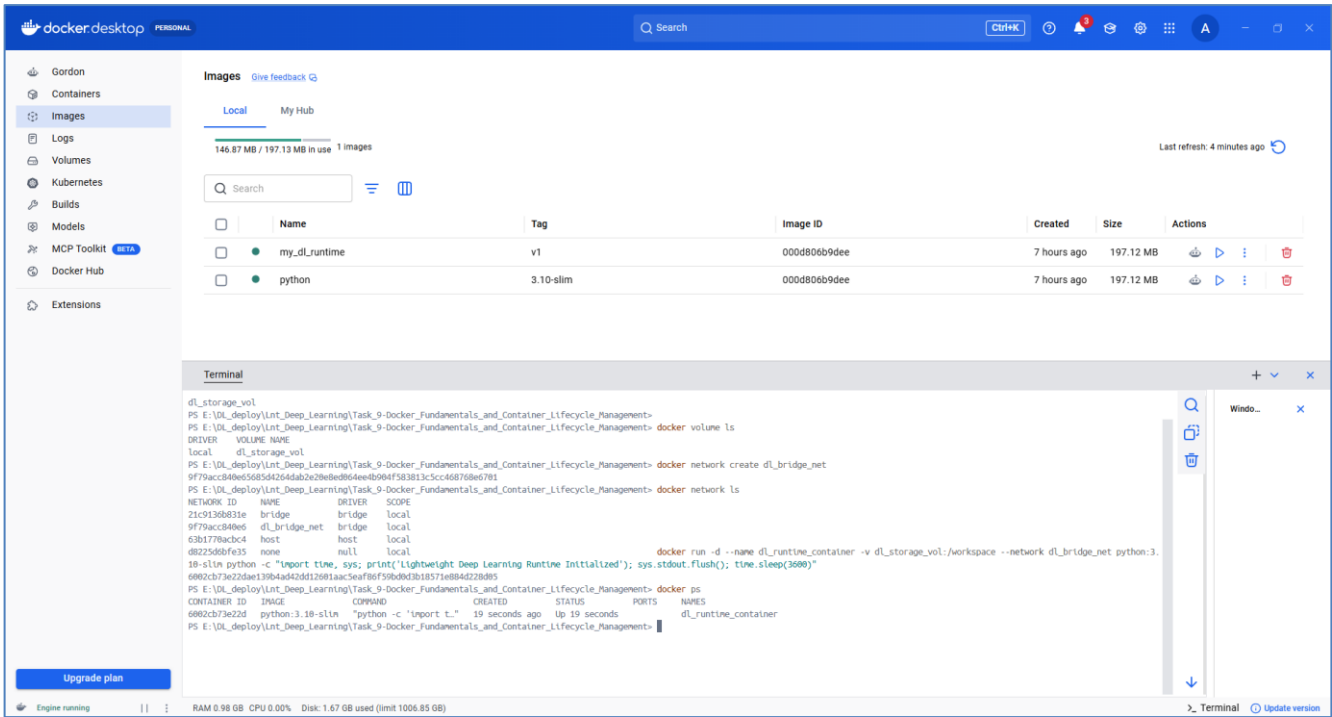
**Result & Technical Observation:** Provisions a host-managed volume (`dl\_storage\_vol`) for persistent model data and log storage, and an isolated bridge network (`dl\_bridge\_net`) for container inter-communication.

## Step 5: Running Container in Detached Mode

### Command Executed:

```
docker run -d --name dl_runtime_container -v dl_storage_vol:/workspace --network
dl_bridge_net python:3.10-slim python -c "import time; time.sleep(3600)"
docker ps
```

Execution Output Log:



5. Running Container Status

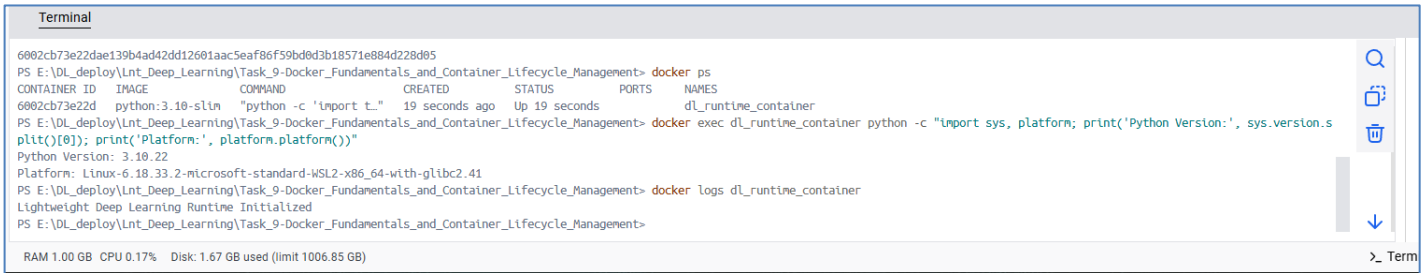
**Result & Technical Observation:** Spawns an isolated container instance in detached mode (`-d`). Mounts persistent volume to `/workspace` and attaches to `dl\_bridge\_net`.

Step 6: Executing In-Container Commands & Viewing Logs

Command Executed:

```
docker exec dl_runtime_container python -c "import sys; print('Python Version:', sys.version.split()[0])"
docker logs dl_runtime_container
```

Execution Output Log:



6.Exec Command & Container Logs

**Result & Technical Observation:** Executes a Python inspection command inside the active container namespace without needing an SSH connection. `logs` fetches stdout/stderr output streams.

**Step 7: Pause, Unpause & Resource Statistics Monitoring**

**Command Executed:**

```
docker pause dl_runtime_container
docker ps
docker unpause dl_runtime_container
docker stats dl_runtime_container --no-stream
```

**Execution Output Log:**

Terminal

PS E:\DL\_deploy\Int\_Deep\_Learning\Task\_9-Docker\_Fundamentals\_and\_Container\_Lifecycle\_Management> docker pause dl\_runtime\_container  
dl\_runtime\_container  
PS E:\DL\_deploy\Int\_Deep\_Learning\Task\_9-Docker\_Fundamentals\_and\_Container\_Lifecycle\_Management> docker ps  
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES  
6002cb73e22d python:3.10-slim "python -c 'import t..." 3 minutes ago Up 3 minutes (Paused) dl\_runtime\_container  
PS E:\DL\_deploy\Int\_Deep\_Learning\Task\_9-Docker\_Fundamentals\_and\_Container\_Lifecycle\_Management> docker unpause dl\_runtime\_container  
dl\_runtime\_container  
PS E:\DL\_deploy\Int\_Deep\_Learning\Task\_9-Docker\_Fundamentals\_and\_Container\_Lifecycle\_Management> docker stats dl\_runtime\_container --no-stream  
CONTAINER ID NAME CPU % MEM USAGE / LIMIT MEM % NET I/O BLOCK I/O PIDS  
6002cb73e22d dl\_runtime\_container 0.00% 9.051MiB / 7.674GiB 0.12% 1.7kB / 126B 9.17MB / 623kB 1  
PS E:\DL\_deploy\Int\_Deep\_Learning\Task\_9-Docker\_Fundamentals\_and\_Container\_Lifecycle\_Management>  
  
RAM 1.05 GB CPU 0.08% Disk: 1.67 GB used (limit 1006.85 GB)

*7.Pause Status & Resource Metrics*

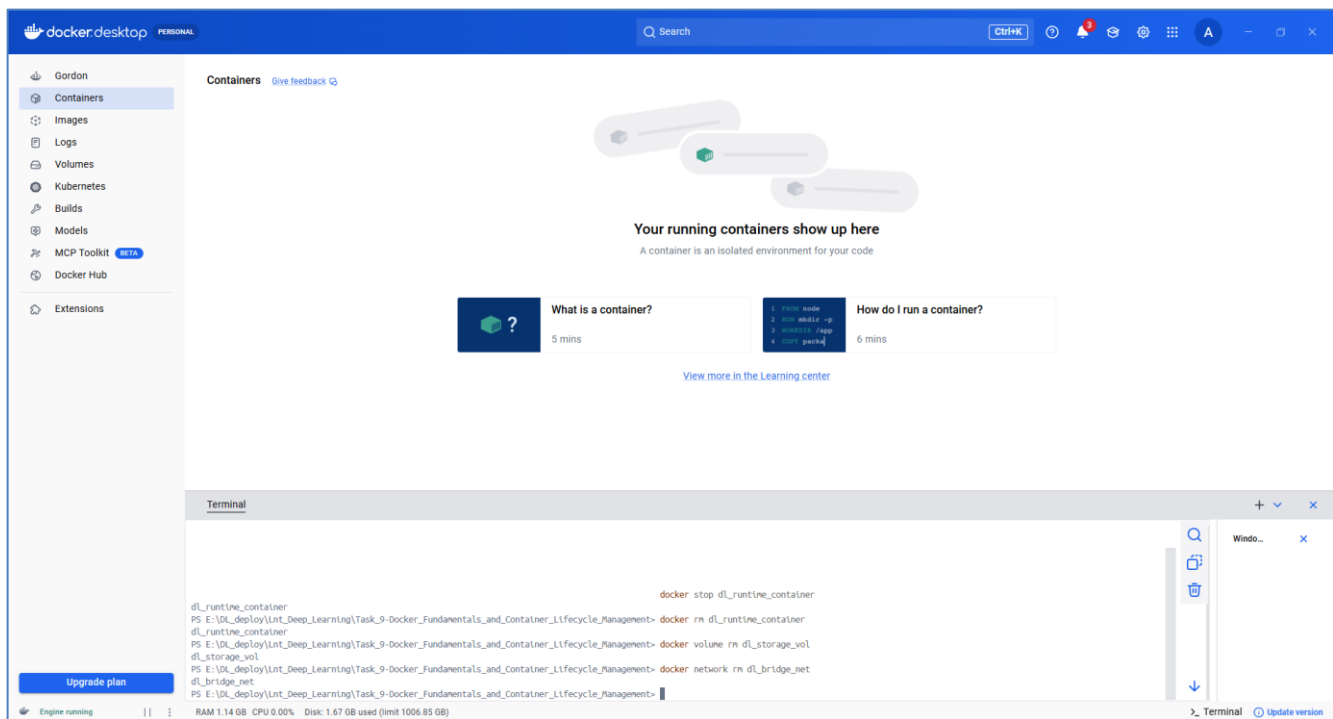
**Result & Technical Observation:** `pause` freezes all processes inside the container using Linux cgroups (status changes to `Up (Paused)`). `stats` measures real-time CPU %, RAM, and I/O consumption.

**Step 8: Container Graceful Stop & Resource Cleanup**

**Command Executed:**

```
docker stop dl_runtime_container
docker rm dl_runtime_container
docker volume rm dl_storage_vol
docker network rm dl_bridge_net
```

**Execution Output Log:**



## 8.Container Shutdown & Cleanup

**Result & Technical Observation:** Sends `SIGTERM` signal to stop container PID 1 cleanly. Removes container metadata, volume, and bridge network to leave Docker engine in a clean state.

## 4. Lifecycle Operations Results & Verification Matrix

| Docker CLI Command    | Lifecycle Phase       | Target Object                   | Verified Result & Outcome                       |
|-----------------------|-----------------------|---------------------------------|-------------------------------------------------|
| docker pull           | Image Acquisition     | Docker Hub (`python:3.10-slim`) | Downloaded image layers (~125 MB)               |
| docker tag            | Image Management      | Local Image Cache               | Created alias tag `my_dl_runtime:v1`            |
| docker volume create  | Storage Setup         | Named Storage Volume            | Created persistent volume `dl_storage_vol`      |
| docker network create | Networking Setup      | Bridge Network                  | Created isolated bridge network `dl_bridge_net` |
| docker run            | Container Creation    | Container Instance              | Spawned detached container with volume & net    |
| docker exec           | Interactive Execution | Container Namespace             | In-container Python version                     |

|                        |                       |                         |                                            |
|------------------------|-----------------------|-------------------------|--------------------------------------------|
|                        |                       |                         | verified                                   |
| docker logs            | Logging & Diagnostics | Container Stdout/Stderr | Fetches stdout log stream                  |
| docker pause / unpause | Process Control       | Linux cgroups           | Successfully frozen and resumed container  |
| docker stats           | Monitoring            | Resource Utilization    | Retrieved live CPU %, RAM, Net I/O metrics |
| docker stop / rm       | Cleanup Operations    | Container Metadata      | Graceful SIGTERM exit & container purge    |

## 5. Conclusion

Task 9 demonstrated comprehensive mastery of Docker fundamentals and container lifecycle operations. By leveraging lightweight base images (`python:3.10-slim`, ~125 MB), provisioning host-managed storage volumes, and isolating network interfaces, deep learning applications can be portably packaged and efficiently managed across MLOps deployment pipelines.

### GitHub:

[https://github.com/AdityaBahira/Lnt\\_Deep\\_Learning/tree/main/Task\\_9-Docker\\_Fundamentals\\_and\\_Container\\_Lifecycle\\_Management](https://github.com/AdityaBahira/Lnt_Deep_Learning/tree/main/Task_9-Docker_Fundamentals_and_Container_Lifecycle_Management)